

# Sesiunea 4: Manipularea DOM-ului

## Capitolul 1: Conectarea JavaScript cu HTML-ul (Obiectul document și Selectoarele JS)

### Obiective de învățare

- 1.1 document - Puntea de legătură (The Bridge)
- 1.2 Instrumentul Suprem: document.querySelector()
- 1.3 querySelectorAll() - Când vrem lista completă
- 1.4 Știați că...?
- 1.5 Exerciții Practice
- 1.6 Soluții la Exerciții
- 1.7 🤖 Mentalitatea de Inginer în Era AI: Locul tău pe piață

## Capitolul 2: Dăm viață paginii - Ascultătorii de evenimente (addEventListener)

### Obiective de învățare

- 2.1 Ce este un Eveniment (Event)?
- 2.2 Sintaxa: addEventListener
- 2.3 Tipuri comune de evenimente pentru QA
- 2.4 Știați că...?
- 2.5 Exercițiu Practic (Cap-Coadă)
- 2.6 Cum se traduce asta în munca unui QA?
- 2.7 🤖 Mentalitatea de Inginer în Era AI: Locul tău pe piață

## Capitolul 3: Adăugarea și Ștergerea Dinamică a Elementelor în DOM

### Obiective de învățare

- 3.1 Nașterea unui Element: document.createElement()
- 3.2 Inserarea în Pagină: appendChild()
  - Ciclul de Viață al unui Element Dinamic
- 3.3 Distrugerea unui Element: remove()
- 3.4 Workshop Practic: Task Tracker Funcțional!
- 3.5 🤖 Mentalitatea de Inginer în Era AI: "Acul care se mișcă în carul cu fân"

## Capitolul 4: Concepte Avansate (Bonus) - Latența și Propagarea Evenimentelor

### Obiective de învățare

4.1 Inamicul numărul 1 al Automatizării: Latența (setTimeout)

4.2 Cum distruge Latența testele QA?

4.3 Secretul interfețelor: Event Bubbling (Bulele de săpun)

4.4 Oprirea propagării (stopPropagation)

4.5 🤖 Mentalitatea de Inginer în Era AI

## Capitolul 5: Tema Sesiunii 4 & Extra Practice (Spre nivelul de Senior)

5.1 Tema pentru Acasă (Proiect Sesiunea 4)

5.2 Provocarea Bonus (Opțional): Dezactivarea Inteligentă a Butonului

5.3 🗝️ Cheile de Rezolvare (Soluții & Explicații)

# Capitolul 1: Conectarea JavaScript cu HTML-ul (Obiectul document și Selectoarele JS)

## Obiective de învățare



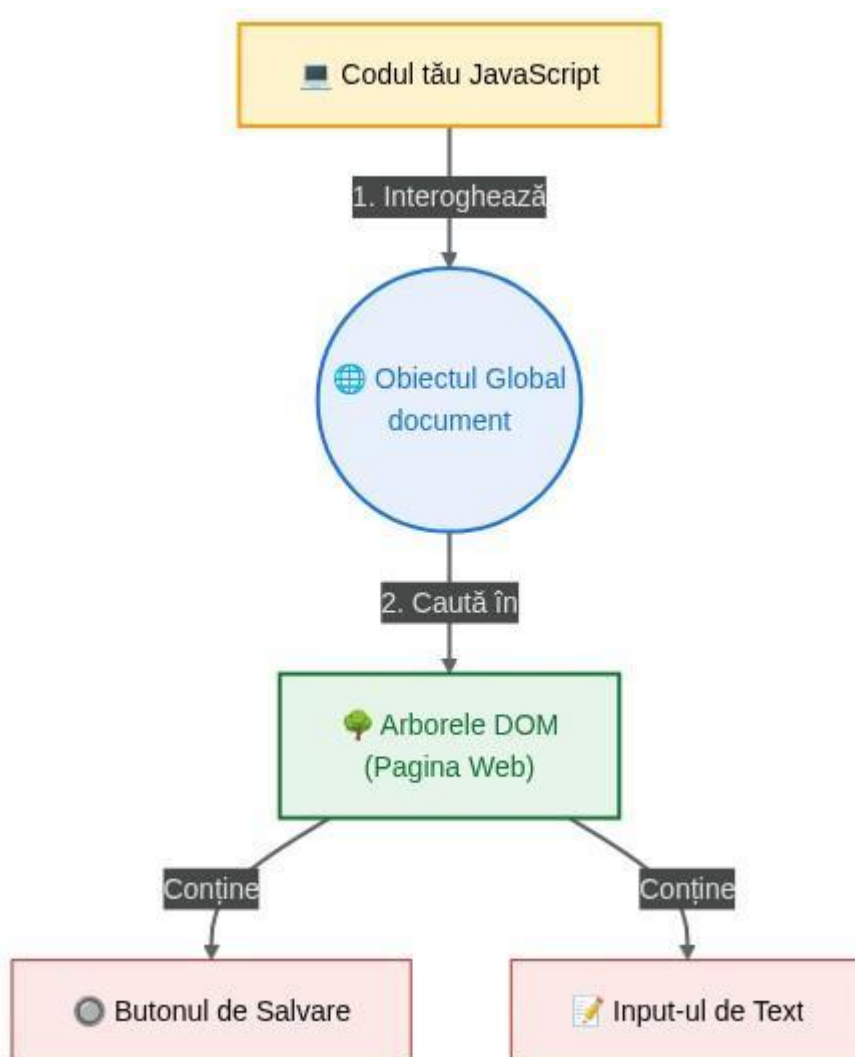
- Înțelegi rolul obiectului global document ca punte de legătură între codul tău JS și structura vizuală HTML.
- Folosești comanda document.querySelector() pentru a găsi elemente specifice în pagină, bazându-te pe selectoarele CSS învățate anterior.
- Faci diferența între selectarea unui singur element și selectarea unei liste de elemente cu document.querySelectorAll().
- Descoperi paralela directă între selecția nativă din JavaScript și comenzile pe care le vei folosi mai târziu în Playwright (ex: page.locator()).

## 1.1 document - Puntea de legătură (The Bridge)

Până acum (în Sesiunea 3), am rulat cod JavaScript în consolă pentru a face calcule matematice, pentru a lucra cu variabile și funcții. Dar codul nostru era oarecum orb. Nu știa absolut nimic despre butoanele sau textele din fișierul nostru HTML.

Când browserul încarcă pagina ta HTML, el o transformă într-o structură de date ascunsă numită **DOM (Document Object Model)**. Vestea bună? Browserul creează automat o variabilă globală în JavaScript numită **document**.

Această variabilă este "cheia universală" a aplicației tale. Ea reprezintă și conține întreaga pagină web!



**Fig. 1** - Diagrama ilustrează arhitectura interacțiunii dintre scriptul JavaScript și elementele unei pagini web. **Codul JavaScript** accesează structura HTML prin



intermediul **obiectului global document**, care servește ca interfață de programare. Acesta interoghează **arborele DOM** pentru a identifica, citi și manipula elementele dinamice finale, precum **Butonul de Salvare** și **Input-ul de Text**.

## 1.2 Instrumentul Suprem: **document.querySelector()**

Cum îi spunem lui JavaScript să "pună mâna" pe un buton ca să putem interacționa cu el mai târziu? Folosind exact "Selectoarele CSS" pe care le-ai învățat în Sesiunea 2 (clase, ID-uri, attribute)!

Cea mai importantă și modernă metodă este `document.querySelector('selector_css')`. Aceasta caută în pagină (de sus în jos) și returnează **primul element** care se potrivește cu descrierea ta.

Exemple de utilizare:

```
// Găsim un element după ID (Recomandat)
const butonAaugare = document.querySelector('#add-task-btn');

// Găsim un element după Clasă (Va aduce doar primul găsit!)
const primulTask = document.querySelector('.task-item');

// Găsim un element după un Atribut de Test (Stilul QA Automation)
const butonStergere = document.querySelector('[data-testid="delete-btn"]');

// Afișăm în consolă ce am găsit, pentru a fi siguri
console.log(butonAaugare);
```

🐶 **Analogie de QA:** `querySelector` este ca un câine de urmă. Îi dai să miroasă o haină (selectorul CSS), iar el aleargă prin tot blocul (DOM) și îți aduce înapoi primul om care se potrivește.

## 1.3 querySelectorAll() - Când vrem lista completă

Ce se întâmplă dacă vrei să numeri câte task-uri ai în aplicație? Dacă folosești `querySelector`, vei primi doar primul task, ignorându-le pe restul.

Pentru a primi o listă completă (foarte asemănătoare cu un `Array`), folosim `document.querySelectorAll()`.

```
// Aducem TOATE elementele care au clasa .task-item
const listaToateTaskurile = document.querySelectorAll('.task-item');

// Acum putem folosi proprietatea .length (exact ca la Array-uri)
console.log("Avem un număr total de task-uri: " +
listaToateTaskurile.length);
```

Acest concept este esențial pentru testele viitoare. În Playwright, când verifici dacă într-un coș de cumpărături s-au adăugat exact 3 produse, framework-ul execută în fundal un mecanism foarte asemănător pentru a număra elementele din DOM!

## 1.4 Știați că...?



- **\$0 în DevTools:** Există un truc secret în Consola Chrome. Dacă mergi în tab-ul *Elements*, dai click pe orice cod HTML, apoi treci în tab-ul *Console* și scrii pur și simplu `$0` și apeși Enter, JavaScript îți va returna instantaneu acel element fără să mai scrii niciun `querySelector`! Este genial pentru debugging rapid.
- **Metodele Legacy:** Pe site-uri vechi (sau în tutoriale din 2010), vei vedea des metode precum `document.getElementById('id')` sau `document.getElementsByClassName('clasa')`. Deși funcționează perfect și sunt puțin mai rapide la executare, industria modernă a standardizat folosirea lui `querySelector` pentru că îți permite să scrii interogări mult mai complexe și înlănțuite (ex: `.container > button:first-child`), folosind o singură comandă unificată.

## 1.5 Exerciții Practice

Deschide fișierul `index.html` al aplicației tale "Task Tracker" în browser, apasă `F12` și mergi la tab-ul `Console`.

### Misiunea 1:

Folosește `document.querySelector` pentru a găsi câmpul de text unde utilizatorul scrie numele unui task (input-ul de text). Salvează-l într-o constantă numită `inputTask` și printează-l în consolă cu `console.log`.

### Misiunea 2:

Folosește `document.querySelectorAll` pentru a găsi absolut toate tag-urile `<button>` de pe pagina ta. Câte butoane ai în total în aplicație la acest moment?

## 1.6 Soluții la Exerciții

### Soluție Misiunea 1:

```
// Presupunând că input-ul tău are id-ul "task-name" sau clasa "task-input"
const inputTask = document.querySelector('#task-name');
console.log(inputTask);
```

### Soluție Misiunea 2:

```
const toateButoanele = document.querySelectorAll('button');
console.log(toateButoanele.length); // Îți va printa un număr, ex: 3
```

## 1.7 Mentalitatea de Inginer în Era AI: Locul tău pe piață

### QA vs. AI: Tranziția către Gândirea de Automatizare

- **Ce face AI-ul bine:** Dacă îi ceri unui AI (ex: ChatGPT, GitHub Copilot) să scrie cod pentru a manipula o pagină, el îți va genera cod instantaneu.
- **Capcana (Unde AI-ul eșuează):** AI-ul a fost antrenat pe milioane de linii de cod vechi

de peste un deceniu. Dacă nu ești specific, AI-ul îți va genera adesea selectoare bazate pe `document.getElementsByTagName` sau se va lega de clase CSS specifice designului (ex: `.bg-blue-500`) pentru a găsi elementele. Această abordare generează scripturi "fragile", care se vor rupe imediat ce UI-ul suferă mici modificări vizuale.

- **Cum gândește un inginer QA:** Inginerul QA știe că testabilitatea este rege. El va scrie (sau va instrui AI-ul să scrie) selecții bazate exclusiv pe `querySelector` folosind attribute stabile de test, de tipul `[data-testid="submit-btn"]`. De ce? Pentru că atunci când vei trece la Playwright (Modulul 4), comanda nativă `document.querySelector('[data-testid="submit-btn"]')` se va traduce arhitectural 1:1 în `page.getByTestId('submit-btn')`.
- **De ce vei rămâne relevant pe piață:** Un simplu executant copiază orbește codul oferit de AI. Tu, în schimb, acționezi ca un "Arhitect de Testare". Forțând o anumită metodologie de selectare a elementelor chiar și în JavaScript-ul de bază, tu te asiguri că aplicația și testele vor "vorbi" exact aceeași limbă, reducând drastic timpul de mentenanță pe termen lung pentru întreaga echipă.

# Capitolul 2: Dăm viață paginii - Ascultătorii de evenimente (addEventListener)

## Obiective de învățare

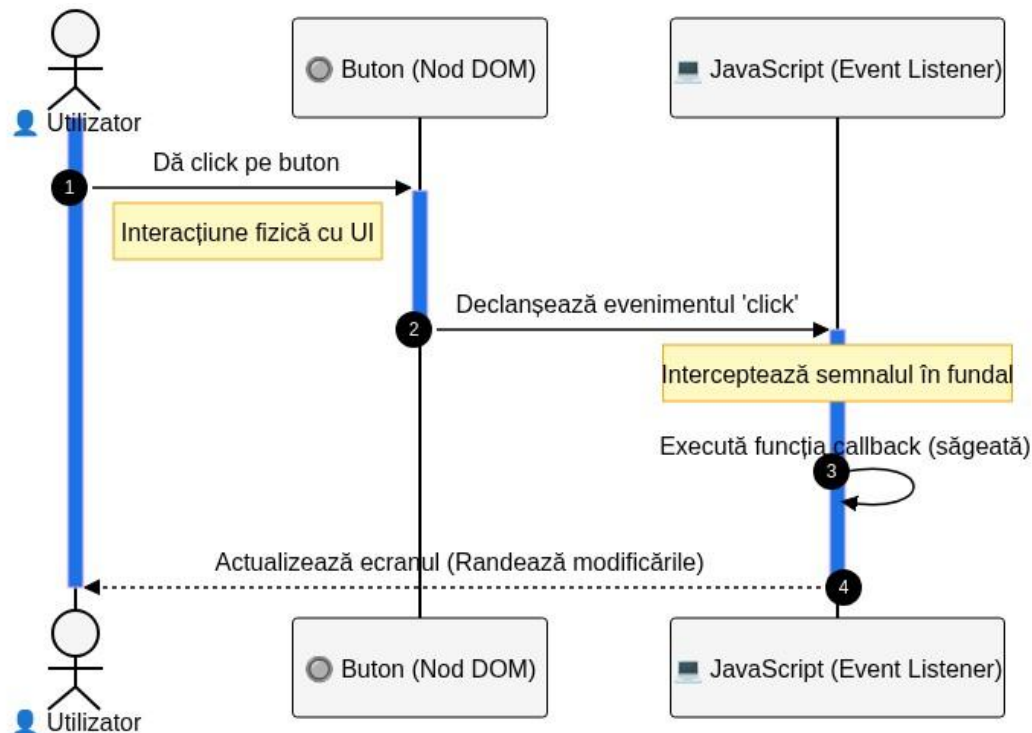
- Înțelegi ce este un eveniment (Event) în contextul unui browser web.
- Folosești metoda `addEventListener` pentru a face elementele HTML să reacționeze la acțiunile utilizatorului.
- Lucrezi cu funcții de tip *Callback* (Arrow Functions) care se execută doar la momentul potrivit.
- Corelezi evenimentele native din JavaScript cu acțiunile de testare pe care le vei executa mai târziu cu Playwright.

## 2.1 Ce este un Eveniment (Event)?

Imediat ce pagina web s-a încărcat, browserul stă și "ascultă" în liniște. Când utilizatorul mișcă mouse-ul, dă un click, apasă o tastă sau face scroll, browserul generează un **Eveniment**.

Fără cod JavaScript care să reacționeze la aceste evenimente, click-urile tale ar lovi elementele HTML fără să se întâmple absolut nimic (în afară de elementele native, cum sunt link-urile `<a>`).

Aici intră în scenă **Ascultătorul de Evenimente (Event Listener)**. Acesta este ca un agent de pază pe care îl pui lângă un buton și căruia îi spui: *"Stai aici și urmărește acest buton. Când cineva dă click pe el, declanșează imediat alarma (funcția mea)!"*



**Fig. 2** - Diagrama de secvență ilustrează fluxul asincron de gestionare a evenimentelor în browser (*Event Handling*). Procesul începe cu **interacțiunea utilizatorului** (click), moment în care **elementul DOM** capturează acțiunea și emite un semnal către **JavaScript**. Scriptul interceptează asincron evenimentul prin structura de tip *Event Listener*, execută funcția de răspuns (*callback*) și reflectă instantaneu modificările rezultate în interfața grafică.

## 2.2 Sintaxa: addEventListener

Pentru a adăuga această interactivitate, folosim o metodă (o comandă) atașată direct de elementul pe care l-am selectat din DOM în capitolul anterior. Această metodă are nevoie de două "ingrediente" (parametri):

- **Numele evenimentului** (un String, ex: 'click').
- **Ce să se întâmple** (o Funcție Săgeată, numită și funcție *Callback*, care deține codul de executat).

```
// Pasul 1: Găsim elementul în pagină
const butonSalvare = document.querySelector('#add-task-btn');

// Pasul 2: Îi atașăm un "ascultător"
```

```
butonSalvare.addEventListener('click', () => {  
    // Tot ce scriem între aceste acolade se va executa DOAR când se dă  
    click!  
    console.log("Ai apăsător pe butonul de salvare!");  
});
```

● **Observație importantă:** Funcția Săgeată de mai sus nu se execută imediat cum JavaScript citește fișierul. Ea stă "în așteptare" în memorie, uneori la nesfârșit, până când utilizatorul realizează acțiunea de click.

## 2.3 Tipuri comune de evenimente pentru QA

În automatizare, vei simula acțiunile unui om. Iată cele mai folosite evenimente pe care dezvoltatorii le ascultă și pe care tu le vei declanșa cu robotul:

- 'click': Lansat la apăsarea unui buton sau a unui link.
- 'input': Lansat de fiecare dată când valoarea dintr-un câmp de text (<input>) se modifică (la fiecare literă tastată). În Playwright, acțiunea echivalentă care declanșează acest eveniment este `page.fill()`.
- 'change': Similar cu input, dar de obicei lansat la final (când utilizatorul iese din câmpul de text) sau când selectează o opțiune dintr-un meniu drop-down (<select>).
- 'keydown' / 'keyup': Lansat când o tastă fizică este apăsată (ex: apăsarea tastei Enter pentru a trimite un formular).

## 2.4 Știați că...?



- **Playwright nu "dă click" magic:** Când tu îi spui lui Playwright `page.click('#btn')`, el nu trimite un deget invizibil pe ecran. "Sub capotă", el trimite un semnal direct motorului browser-ului pentru a emite artificial (dispatch) exact acest eveniment de 'click'. Dacă dezvoltatorul a uitat să pună `addEventListener` pe buton, testul tău Playwright va "da click" cu succes (nu va da eroare), dar nu se va întâmpla nimic în aplicație!
- **Atenție la atributele Inline (Antimodelul):** Poate ai văzut cod HTML vechi care arată așa: `<button onclick="salveaza()">`. Aceasta se numește "Inline Event Handler". Este o practică învechită și evitată astăzi, deoarece amestecă structura (HTML) cu logica (JavaScript), încălcând principiul de Separare a Preocupărilor (*Separation of*

Vom manipula vizual DOM-ul aplicației noastre "Task Tracker". Fă acești pași direct în browser!

**Scenariul:** Vrem ca atunci când utilizatorul apasă butonul "Salvează Task", butonul să își schimbe textul în "Salvat!" și culoarea în verde, pentru a oferi feedback vizual.

## Pasul 1: Pregătirea terenului

Deschide fișierul tău index.html (din Sesiunea 1) în Chrome. Apasă F12 și mergi la tab-ul **Console**.

- Codul HTML complet ([index.html](#))

Acesta este scheletul din Sesiunea 1, la care am adăugat linia `<script src="app.js"></script>` jos, fix înainte de închiderea `</body>`, pentru a lega logica.

```

    }
  </style>
</head>
<body>
  <header>
    <h1>Task Tracker Pro - QA Edition</h1>
  </header>
  <main>
    <!-- Secțiunea de adăugare task -->
    <section class="task-input-section">
      <h2>Adaugă un Task Nou</h2>
      <form id="add-task-form">
        <label for="task-name">Nume Task:</label>
        <input type="text" id="task-name" data-testid="input-task-
name" placeholder="Ex: Scrie scenariu E2E" required>
        <br>
        <br>
        <!-- Butonul țintă pentru exercițiul nostru -->
        <button type="submit" id="add-task-btn" data-testid="submit-
new-task">Salvează Task</button>
      </form>
    </section>
    <section class="task-list-section">
      <h2>Task-uri Active</h2>
      <!-- Acesta este elementul CRITIC pe care îl caută JavaScript-ul
tău! -->
      <ul id="active-tasks-list">
        <!-- Aici vor apărea dinamic noile <li>-uri adăugate de tine -
->
      </ul>
    </section>
  </main>
  <!-- Legătura către fișierul JavaScript -->
  <script src="app.js"></script>
</body>
</html>

```

## 2. Codul JavaScript complet (app.js)

Acest cod conține exercițiul exact așa cum a fost gândit, pus într-un fișier separat pentru a respecta bunele practici (Separation of Concerns). Dacă se dorește rularea doar în DevTools

Console, putem pur și simplu să dăm copy-paste acestui bloc direct acolo.

```
// PASUL 1: Găsim elementul pe pagină (Selectarea)
const butonSalvare = document.querySelector('#add-task-btn');

// PASUL 2: "Ascultăm" interacțiunea utilizatorului
// Parametrul 'event' (sau 'e') prinde detaliile acțiunii fizice de click
butonSalvare.addEventListener('click', (event) => {

    // TRUC VITAL: Oprim comportamentul nativ al formularului!
    // Dacă nu punem asta, pagina își dă refresh instant și pierdem starea.
    event.preventDefault();

    // PASUL 3: Modificăm DOM-ul live (Schimbăm Textul)
    butonSalvare.textContent = "Task Salvat! ☒";

    // PASUL 4: Modificăm CSS-ul prin JavaScript (Schimbăm Culoarea)
    butonSalvare.style.backgroundColor = "#28a745"; // Verde de succes
    butonSalvare.style.transform = "scale(1.1)";    // Îl facem puțin mai mare pentru efect

    // Raportăm în consolă pentru QA
    console.log("Interacțiune detectată! Funcția a oprit refresh-ul și a modificat DOM-ul.");
});
```

## Pasul 2: Selectarea butonului

În consolă, scrie următoarea linie și apasă Enter pentru a prinde butonul într-o constantă:

```
const btnAdauga = document.querySelector('[data-testid="submit-new-task"]');
```

(Dacă nu ai pus atributul `data-testid`, folosește `querySelector('#add-task-btn')`).

## Pasul 3: Adăugarea Ascultătorului și a Logicii (Execuția)

Scrie (sau copiază) următorul bloc de cod în consolă și apasă Enter. Atenție, codul folosește obiectul `.style` pentru a modifica direct CSS-ul din JavaScript!

```
btnAdauga.addEventListener('click', () => {
    // 1. Schimbăm textul din interiorul butonului
```

```

    btnAdauga.textContent = "Task Salvat! ✓";

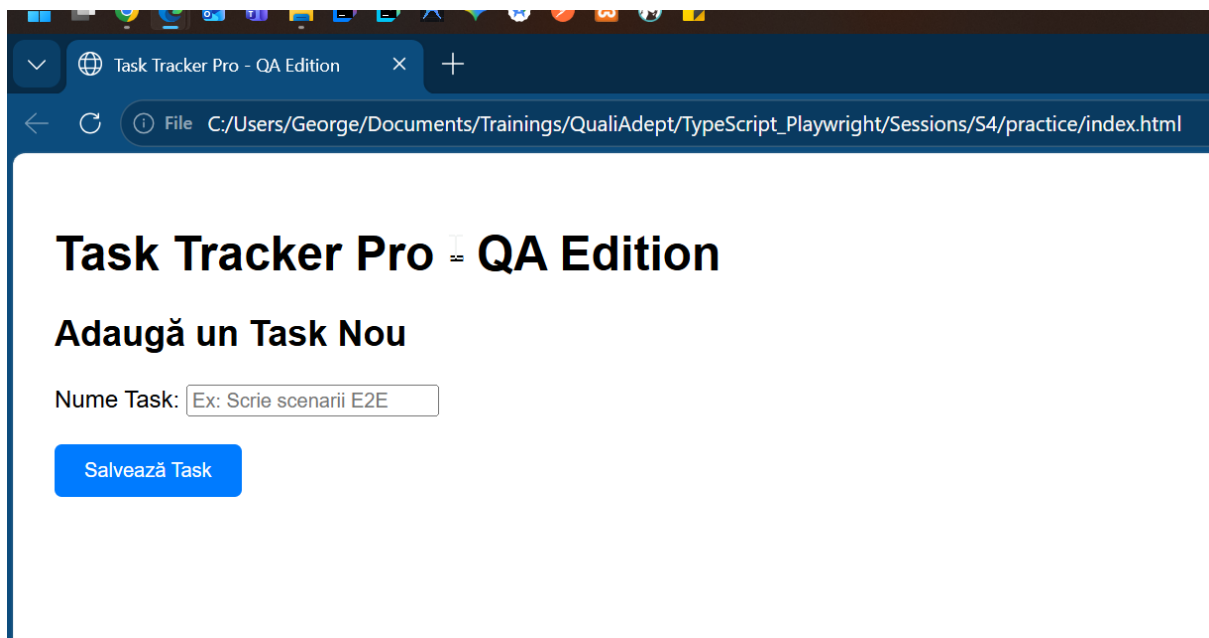
    // 2. Modificăm proprietatea CSS de fundal (observă scrierea
    camelCase)
    btnAdauga.style.backgroundColor = "#28a745";

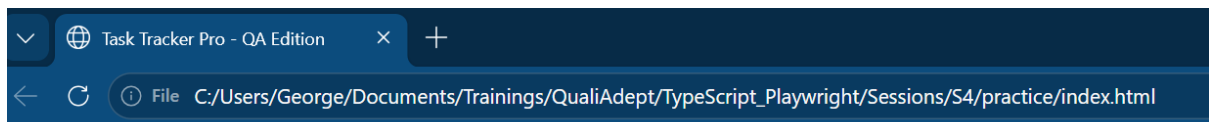
    // 3. Schimbăm culoarea textului în alb pentru contrast
    btnAdauga.style.color = "white";
  });

```

#### Pasul 4: Testarea Manuală (Momentul de Magie!)

După ce ai dat Enter în consolă (browserul va afișa undefined, e normal), închide DevTools. Du-te cu mouse-ul și **dă click** fizic pe butonul tău din pagină. Ai observat cum interfața a reacționat instantaneu și s-a colorat în verde? Tocmai ai scris prima ta logică de interacțiune Frontend!





## Task Tracker Pro - QA Edition

### Adaugă un Task Nou

Nume Task:

Task Salvat! ✓

## 2.6 Cum se traduce asta în munca unui QA?

Exercițiul de mai sus reprezintă exact ceea ce face un developer Front-End. Rolul tău ca inginer de automatizare este să verifici dacă acest `addEventListener` a funcționat. Mai târziu în curs, testul tău Playwright pentru acest scenariu va arăta fix așa:

```
// Așa va arăta testul tău E2E în Modulul 4!  
await page.getByTestId('submit-new-task').click();  
await expect(page.getByTestId('submit-new-task')).toHaveText('Task  
Salvat! ✓');
```

## 2.7 🧠 Mentalitatea de Inginer în Era AI: Locul tău pe piață

### 🔑 QA vs. AI: Capcana "onClick" și Testabilitatea

- **Ce face AI-ul bine:** Când îi ceri unui asistent de programare (ex: GitHub Copilot) să adauge interactivitate unui buton într-un proiect, o va face rapid.
- **Capcana (Unde AI-ul eșuează):** Uneori, mai ales pe proiecte de tip prototip, un AI leneș îți va altera direct fișierul HTML și va adăuga atributul `<button onclick="trimiteFormular()>` (Inline events), în loc să folosească arhitectura modernă cu `addEventListener` în fișierul JS.

- **Cum gândește un inginer QA:** Inginerul se uită imediat pe cod (Code Review) și respinge această abordare. El știe că logica amestecată cu UI-ul face aplicația mult mai greu de testat. Dacă un event listener este atașat curat în JavaScript, poți "moca" (simula/intercepta) funcția trimiteFormular() mult mai ușor în timpul testelor de Componentă. Inginerul va cere refactorizarea codului pentru a menține aplicația curată și la standarde Enterprise.
- **De ce vei rămâne relevant pe piață:** Valoarea ta nu este de a scrie repede o funcție care dă "click" (AI-ul face asta într-o secundă). Valoarea ta este să aperi **arhitectura** și **bunele practici**. Aplicațiile care ocolesc standardele devin cu timpul o "mlaștină" imposibil de menținut. Ca QA, tu ești vocea calității încă de la nivelul codului sursă.

# Capitolul 3: Adăugarea și Ștergerea Dinamică a Elementelor în DOM

## Obiective de învățare



- Înțelegi diferența dintre un element creat în memoria JavaScript și un element vizibil pe ecran.
- Folosești `document.createElement()` pentru a genera elemente HTML noi din cod.
- Injectezi elementele noi în structura paginii folosind `appendChild()` sau `append()`.
- Elimini elemente din pagină folosind metoda `remove()`.
- Corelezi aceste concepte cu provocările reale din testarea automatizată (elemente care apar cu întârziere).

## 3.1 Nașterea unui Element: `document.createElement()`

În aplicațiile web moderne (scrise în React sau Angular), HTML-ul inițial este adesea aproape gol. Interfața este construită "din mers" (dinamic) de către JavaScript, pe măsură ce datele vin de la server sau utilizatorul interacționează cu pagina.

Pentru a crea un element nou direct din JavaScript, folosim comanda `document.createElement('nume_tag')`.

```
// 1. Creăm un element de tip listă <li> în MEMORIA browserului
const taskNou = document.createElement('li');

// 2. Îl configurăm (îi dăm text, clase, attribute)
taskNou.textContent = "Învăță Playwright";
taskNou.className = "task-item";
```

**Atenție:** În acest moment, elementul `taskNou` există doar în memoria temporară a JavaScript-ului. El **NU** este vizibil pe ecran, pentru că nu a fost atașat în "Arborele Genealogic" al DOM-

ului.

## 3.2 Inserarea în Pagină: appendChild()

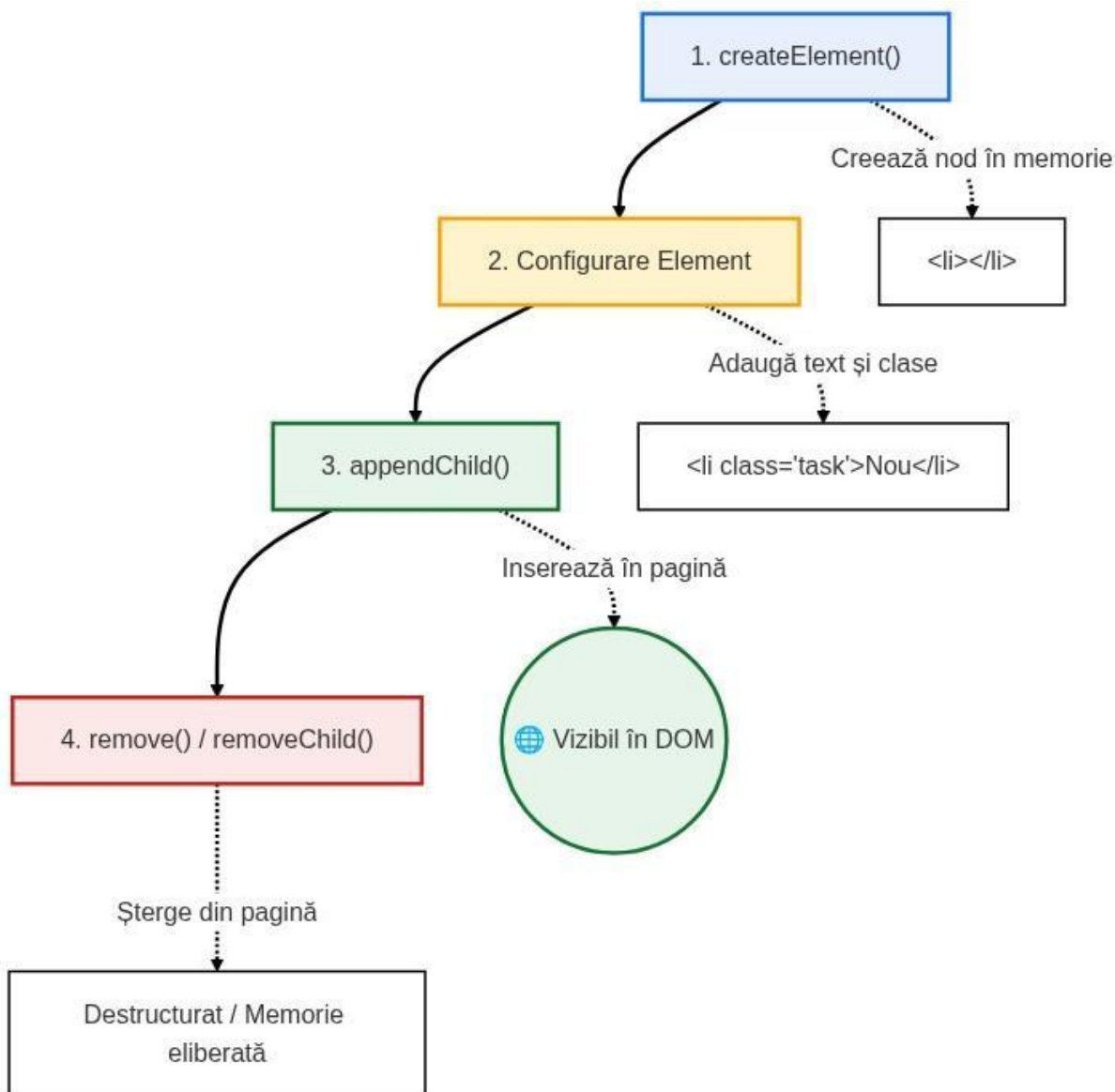
Pentru a face elementul vizibil, trebuie să îl agățăm de un "Părinte" existent deja în HTML. Gândește-te la asta ca la asamblarea unui joc Lego: întâi construiești piesa nouă în mână (createElement), apoi o înfigi pe placa de bază (appendChild).

```
// Găsim părintele în pagină (ex: lista noastră de task-uri <ul>)  
const listaTaskuri = document.querySelector('#active-tasks-list');  
  
// Atașăm elementul nou la finalul listei copilului  
listaTaskuri.appendChild(taskNou);
```

Acum, magia s-a produs! Elementul a apărut pe ecran instantaneu, fără ca pagina să își dea refresh.

## Ciclul de Viață al unui Element Dinamic

Iată vizualizarea procesului complet, de la idee până la afișarea pe ecran:



**Fig. 3** - Diagrama detaliază ciclul complet de viață al unui nod HTML gestionat dinamic prin JavaScript. Fluxul cuprinde patru etape: **crearea inițială în memoria RAM** (`createElement`), **configurarea atributelor** (clase CSS, text intern), **atașarea în structura vizibilă a paginii** (`appendChild`) și, în final, **eliminarea nodului din interfața grafică** (`remove` sau `removeChild`). Această ultimă etapă întrerupe legătura elementului cu arborele DOM și permite browserului să elibereze resursele de memorie asociate.

## 3.3 Distrugerea unui Element: `remove()`

Ștergerea unui element este mult mai simplă decât crearea lui. Dacă ai referința către acel element, apelezi pur și simplu metoda `.remove()`.

```
// Găsim primul buton de ștergere de pe pagină
const butonSterge = document.querySelector('.delete-btn');

// Îl eliminăm complet din DOM
butonSterge.remove();
```

**Nota QA:** În automatizare, verificarea ștergerii este o asertare critică (ex: `expect(locator).not.toBeVisible()`).

## 3.4 Workshop Practic: Task Tracker Funcțional!

Hai să unim tot ce am învățat în Sesiunea 4 (Selectare + Evenimente + DOM Dinamic). Vom face formularul tău să adauge efectiv task-uri reale în listă!

*Deschide fișierul tău `app.js` și înlocuiește codul vechi cu acesta. Apoi testează aplicația în browser!*

```
// 1. Selectăm elementele de care avem nevoie
const formular = document.querySelector('#add-task-form');
const inputTask = document.querySelector('#task-name');
const listaTaskuri = document.querySelector('#active-tasks-list');

// 2. Ascultăm evenimentul de 'submit' (trimitere) al formularului
formular.addEventListener('submit', (event) => {
  // Oprim reîncărcarea paginii
  event.preventDefault();

  // Extragem valoarea (textul) scrisă de utilizator în input
  const textNou = inputTask.value;

  // 3. Creăm un element nou <li>
  const liNou = document.createElement('li');
  liNou.className = 'task-item';
  liNou.textContent = textNou;

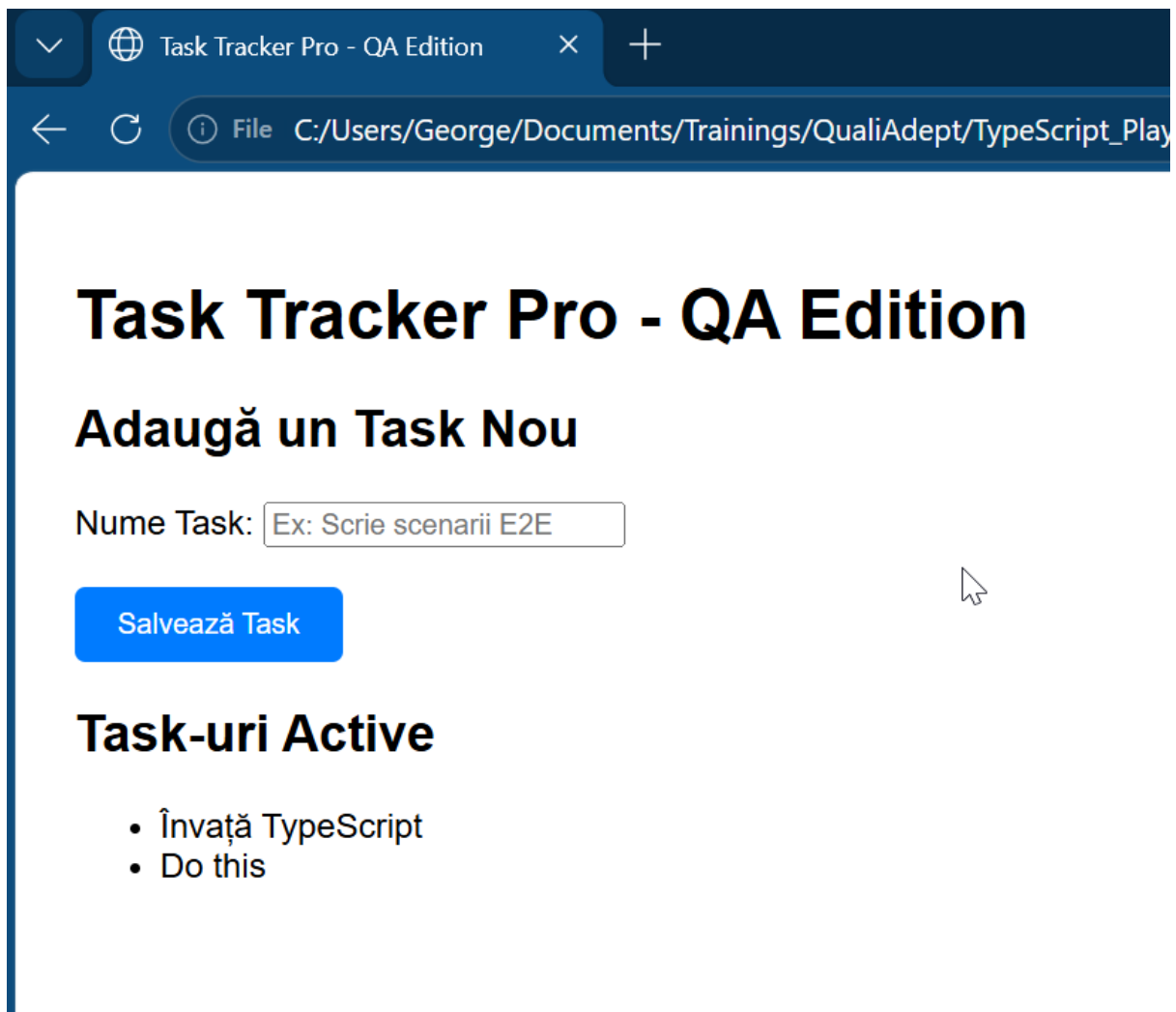
  // 4. Îl injectăm în DOM (în interiorul <ul>)
  listaTaskuri.appendChild(liNou);

  // 5. Curățăm input-ul pentru a face loc următorului task
  inputTask.value = '';

  console.log("Task adăugat cu succes: " + textNou);
});
```

**Testează-!:** Scrie "Învăță TypeScript" în căsuță și apasă "Salvează Task". Vei vedea cum lista

se mărește live, sub ochii tăi! Aplicația ta prinde contur real.



### 3.5 🤖 Mentalitatea de Inginer în Era AI: "Acul care se mișcă în carul cu fân"

#### 🔑 QA vs. AI: Iluzia unui DOM Static

- **Ce face AI-ul bine:** Asistenții bazați pe LLM înțeleg perfect cum să scrie o funcție care dă click pe un element. Ei presupun logic: *Găsește elementul X -> Execută acțiunea Y.*
- **Capcana (Unde AI-ul eșuează):** AI-ul abordează o pagină web ca pe un document PDF (ceva static și neschimbat). Dar tu tocmai ai văzut cum elementele se creează și se șterg în milisecunde (createElement, remove). Dacă un test generat de AI încearcă să valideze textul unui task imediat după ce a dat click pe "Salvează", testul poate pica

brusc (Flaky Test). De ce? Pentru că AI-ul nu a luat în calcul cele 50 de milisecunde în care JavaScript-ul se execută pentru a injecta elementul în DOM cu `appendChild`. Scriptul a fost prea rapid pentru interfață!

- **Cum gândește un inginer QA:** Inginerul știe că DOM-ul este un organism viu. Când un inginer QA scrie un test E2E (sau corectează codul AI-ului), el nu presupune că elementul este acolo. El folosește metode cu **Așteptare Activă (Auto-Waiting)**. În Playwright, tu vei scrie comenzi de tipul: „*Așteaptă până când elementul cu textul X este atașat în DOM (State: attached), și abia apoi validează-l*”.
- **De ce vei rămâne relevant pe piață:** Diferența dintre un tester mediocru și un Senior Automation Engineer este capacitatea de a stabili teste. Înțelegând exact mecanica prin care framework-urile Frontend adaugă elemente în pagină, vei ști de ce testele tale pică și, mai important, cum să scrii asertări reziliente la latențele de randare ale browserului.

## Capitolul 4: Concepte Avansate

# (Bonus) - Latența și Propagarea Evenimentelor

## Obiective de învățare



- Înțelegi fenomenul de latență (delay) în aplicațiile web folosind `setTimeout`.
- Conștientizezi cum latența cauzează teste "Flaky" (instabile) și de ce asertările stricte pică.
- Descoperi conceptul de "Event Bubbling" (propagarea evenimentelor) în DOM.
- Corelezi aceste mecanisme avansate JavaScript cu funcțiile de Auto-Waiting din Playwright.

## 4.1 Inamicul numărul 1 al Automatizării: Latența (`setTimeout`)

În exemplele noastre de până acum, când dădeai click pe un buton, textul sau elementul nou apărea instantaneu (în zero milisecunde). În lumea reală, lucrurile nu stau așa. Când adaugi un produs în coș pe un magazin online, **browserul** trebuie să **trimită** datele la **server**, să **aștepte confirmarea** și abia apoi să îți **afișeze mesajul** de succes pe ecran. Acest drum poate dura între **100ms** și câteva **secunde**.

Pentru a simula această întârziere (latență) în JavaScript, folosim funcția `setTimeout()`.

```
const butonComanda = document.querySelector('#plaseaza-comanda');

butonComanda.addEventListener('click', () => {
  console.log("Comanda se procesează...");

  // Simulăm răspunsul serverului cu o întârziere de 3 secunde (3000 milisecunde)
  setTimeout(() => {
    butonComanda.textContent = "Comandă Plasată!";
    butonComanda.style.backgroundColor = "green";
  }, 3000);
});
```

```
        console.log("UI-ul a fost actualizat după 3 secunde.");  
    }, 3000);  
});
```

## 4.2 Cum distruge Latența testele QA?

Dacă ai scrie un test E2E clasic (stil vechi, cum era Selenium la începuturi) pentru codul de mai sus, testul tău ar arăta cam așa:

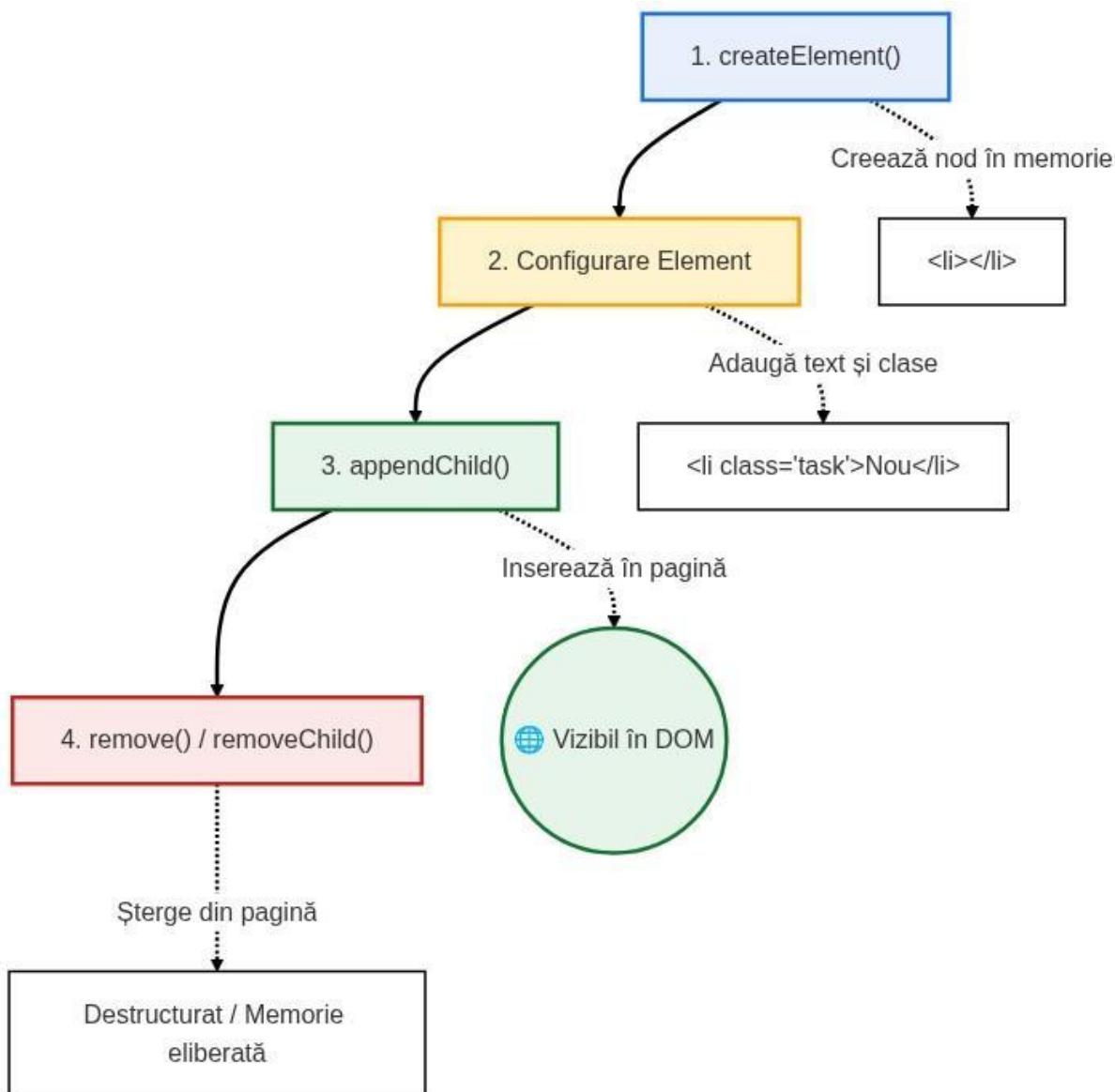
- Dă click pe #plaseaza-comanda
- Validează că textul butonului este "Comandă Plasată!" -> **TEST PICAT! ✗**

*De ce a picat?* Pentru că robotul tău execută pasul 2 în exact 5 milisecunde după pasul 1. Butonul nu a apucat să se facă verde, pentru că serverul (simulat de setTimeout) avea nevoie de 3000 de milisecunde! Acesta este faimosul **Flaky Test** (test care pică și trece aparent aleator, în funcție de viteza internetului).

**Soluția Playwright:** Când vom ajunge la modulul de Playwright, asertările noastre vor arăta așa:

```
await expect(butonComanda).toHaveText("Comandă Plasată!");
```

Cuvântul cheie aici este *Auto-Waiting*. Playwright nu verifică doar o dată și pică. El verifică de zeci de ori pe secundă, așteptând (până la un timeout maxim de 5 secunde) ca acel setTimeout din JavaScript să își termine treaba și să modifice DOM-ul!



**Fig. 4** - Diagrama detaliază ciclul complet de viață al unui nod HTML gestionat dinamic prin JavaScript. Fluxul cuprinde patru etape: **crearea inițială în memoria RAM** (`createElement`), **configurarea atributelor** (clase CSS, text intern), **inserarea în structura web vizibilă** (`appendChild`) și, în final, **eliminarea elementului din interfața grafică** (`remove` sau `removeChild`). Această ultimă etapă întrerupe legătura elementului cu arborele DOM, permițând browserului să elibereze resursele de memorie asociate.

## 4.3 Secretul interfețelor: Event Bubbling (Bulele de săpun)

Imaginați-vă că aveți un buton de ștergere în interiorul unui paragraf, care la rândul lui este în interiorul unui formular.

Ce se întâmplă când dai click pe buton?

În JavaScript, evenimentele "fierb" și se ridică la suprafață exact ca bulele de aer din apă (Event Bubbling).

Când dai click pe <button>, browserul declanșează:

- Click pe <button> (Copil)
- Apoi Click pe <p> (Părinte)
- Apoi Click pe <form> (Bunic)
- Apoi Click pe <body>... până la Rădăcină!

```
// Dacă ai avea ascultători pe fiecare, un singur click ar declanșa 3 acțiuni!  
buton.addEventListener('click', () => console.log("Click pe buton"));  
paragraf.addEventListener('click', () => console.log("Click pe paragraf"));  
formular.addEventListener('click', () => console.log("Click pe formular"));
```

## 4.4 Oprirea propagării (stopPropagation)

Uneori, dezvoltatorii folosesc metoda `event.stopPropagation()` pentru a bloca "bula" să urce mai sus de elementul pe care s-a dat click.

Ca QA, te vei lovi de acest concept când vei încerca să dai click pe un icon mic (ex: o bifă) aflat într-un chenar mare, dar din cauza modului în care a fost gestionată propagarea pe Frontend, click-ul tău s-ar putea să "lovească" doar părintele, iar evenimentul să nu ajungă niciodată la logica dorită, dacă scrii selectorul greșit.

## 4.5 Mentalitatea de Inginer în Era AI

### QA vs. AI: `Thread.sleep()` vs Așteptare Inteligentă

- **Ce face AI-ul bine:** Dacă îi ceri unui AI să scrie un test pentru o pagină unde elementele apar cu întârziere, îți va scrie codul de click.
- **Capcana (Unde AI-ul eșuează):** Un AI prost instruit, atunci când este confruntat cu un test care pică din cauza latenței descrise la secțiunea 4.1, va sugera cea mai toxică

practică din QA: Așteptarea statică (Hard Sleep). Îți va genera o linie de cod care spune `await page.waitForTimeout(3000);` (Oprește testul complet 3 secunde). Dacă ai 200 de teste și AI-ul îți bagă 3 secunde pauză în fiecare, suita ta de teste va rula în 10 minute în loc de 30 de secunde!

- **Cum gândește un inginer QA:** Inginerul știe cum funcționează de fapt `setTimeout` și DOM-ul. El va șterge așteptarea fixă (sleep-ul) și se va asigura că folosește comenzi bazate pe Stare (State-based waiting), cerându-i lui Playwright să acționeze dinamic: *"Așteaptă doar cât este necesar ca DOM-ul să atingă starea corectă".*

# Capitolul 5: Tema Sesiunii 4 & Extra Practice (Spre nivelul de Senior)

## 5.1 Tema pentru Acasă (Proiect Sesiunea 4)

### Sarcina ta (Task-ul de business):

În sesiunile anterioare am pregătit interfața (HTML/CSS) pentru aplicația noastră "Task Tracker", inclusiv un tabel unde vom ține istoricul task-urilor.

Acum a sosit momentul să conectăm JavaScript-ul! Sarcina ta este să interceptezi formularul de adăugare și să injectezi noile task-uri direct ca rânduri noi în tabelul HTML, fiecare venind "la pachet" cu propriul buton funcțional de ștergere.

**Notă pentru execuție:** Vei scrie acest cod în fișierul tău **app.js**. Testează de fiecare dată în browser dând refresh la pagina **index.html**, iar la final validează-ți munca în **platforma QualiAdept!**

### Cerințe Tehnice (Acceptance Criteria pentru platforma de validare):

- **Conectarea fișierului:** Asigură-te că fișierul **app.js** este inclus în **index.html** folosind tag-ul `<script>`.
- **Selectarea Elementelor DOM:** Folosește `document.querySelector(...)` sau `getElementById(...)` pentru a găsi formularul (`#add-task-form`), câmpul de text (`#task-name`) și corpul tabelului (`tbody` sau `#task-list`).
- **Ascultător de Eveniment:** Adaugă un ascultător de eveniment pentru formular folosind `addEventListener('submit', ...)`.
- **Prevenirea Reîncărcării:** În interiorul ascultătorului, apelează metoda `e.preventDefault()` (sau `event.preventDefault()`) pentru a opri reîncărcarea nativă a paginii.
- **Crearea Dinamică:** Extrage textul din input, apoi creează un element de rând nou folosind comanda `document.createElement('tr')`.
- **Structura Rândului:** Asigură-te că noul rând conține celulele necesare (`<td>`), inclusiv un buton de ștergere care are obligatoriu clasa `delete-row`.
- **Inserarea în DOM:** Adaugă noul rând în corpul tabelului folosind metoda `.appendChild(...)` sau `.append(...)`.

- **Curățarea Inputului:** Resetează valoarea câmpului de input la un string gol (`input.value = ''`) după salvarea task-ului.
- **Ștergerea Dinamică:** Implementează logica de ștergere la apăsarea butonului, folosind metoda `.remove()` pe rândul părinte (`<tr>`).

*(Analogie QA: Acest scenariu reprezintă testul suprem de End-to-End: Create & Delete. În Playwright, tu vei completa câmpul, vei da click pe 'Adaugă Task', vei valida că rândul a apărut în tabel, apoi vei da click pe butonul său de 'Șterge' și vei valida cu `expect(locator).not.toBeVisible()` că a dispărut!)*

## 5.2 Provocarea Bonus (Opțional): Dezactivarea Inteligentă a Butonului

### 🔹 Scenariul:

În aplicațiile moderne de top (Enterprise), butonul de "Adaugă Task" este dezactivat vizual (gri, imposibil de apăsăat) atâta timp cât câmpul de text este gol, pentru a preveni trimiterea de task-uri invizibile.

### 🧑‍💻 Misiunea ta:

Găsește butonul de "Adaugă Task" din HTML. Găsește input-ul. Pune un `addEventListener` de tip 'input' pe câmpul de text (acesta se declanșează la fiecare literă tastată de utilizator).

Scrie o logică astfel încât:

- Dacă valoarea input-ului este un string gol (`''`), butonul primește atributul de blocare (folosind `buton.disabled = true;`).
- Dacă input-ul are cel puțin un caracter, butonul devine activ (`buton.disabled = false;`).

## 5.3 🔑 Cheile de Rezolvare (Soluții & Explicații)

🕒 ⚠️ **Nu trișa!** Citește asta doar după ce te-ai blocat sau ai încercat să scrii codul singur **minim 20 de minute**. Codul de mai jos este garantat să treacă de toate validările statice (Regex) ale platformei QualiAdept.

### ✅ Soluție Tema pentru Acasă:

```
// 1. Selectare elemente DOM
const form = document.querySelector('#add-task-form');
```

```
const input = document.getElementById('task-name');
const tbody = document.querySelector('tbody');

// 2 & 3. Ascultător eveniment 'submit' și prevenirea reîncărcării
form.addEventListener('submit', function(e) {
    e.preventDefault();

    // 4. Creăm dinamic elementul de rând nou
    const tr = document.createElement('tr');

    // 5. Generăm structura rândului, incluzând butonul cu clasa 'delete-row'
    tr.innerHTML = '<td>#1003</td><td>' + input.value +
    '</td><td><button class="delete-row">Sterge</button></td>';

    // 6. Adăugăm noul rând în corpul tabelului
    tbody.appendChild(tr);

    // 7. Curățăm inputul după adăugare
    input.value = '';
});

// 8 & 9. Ștergere Dinamică din DOM folosind Event Delegation
document.addEventListener('click', function(e) {
    // Verificăm dacă elementul apăsat are clasa 'delete-row'
    if (e.target.classList.contains('delete-row')) {
        // Căutăm cel mai apropiat părinte de tip <tr> și îl ștergem cu
        .remove()
        e.target.closest('tr').remove();
    }
});
```

## Soluție Provocarea Bonus (Validare Live):

```
// Găsim butonul de submit pe care vrem să-l manipulăm
const butonSubmit = document.querySelector('[data-testid="submit-task-btn"]');

// Setăm starea inițială: când deschidem pagina, câmpul e gol, deci
dezactivăm butonul
butonSubmit.disabled = true;
```

```
// Ascultăm la fiecare literă pe care utilizatorul o tastează în input
input.addEventListener('input', () => {
  // Dacă lungimea textului este 0 (adică e gol), disable rămâne true
  if (input.value.length === 0) {
    butonSubmit.disabled = true;
  } else {
    // Altfel, dacă a scris ceva, activăm butonul
    butonSubmit.disabled = false;
  }
});
```

🎉 **Felicitări masive și aplauze furtunoase!** Prin rezolvarea acestei teme ai finalizat cu succes modulul de Frontend și JavaScript (Sesiunile 1 - 4). Acum deții "cheile" interfeței web. Știi cum sunt construite paginile (HTML/CSS), cum gândesc ele (JS) și cum interacționează elementele între ele.

Următoarea ta oprire? Vom face pasul către **Asincronism** și către **Playwright**! Vei prelua controlul, înlocuind mouse-ul tău cu scripturi de automatizare care vor executa fix aceste acțiuni cu viteza luminii!